Plexus

A blueprint for differentiable tissue: hierarchical sets, operators, and a formal simulation language

Janelia Research Campus

June 18, 2026

Abstract

Living tissues couple biochemical signalling, cellular interaction, mechanical force, and collective dynamics across scales. The frameworks built so far (CellGraph, NeuralGraph, MPMGraph, MetabolismGraph) each implement *one* graph type; a tissue needs them coexisting. This document is a single blueprint, distilled from a working prototype, for building one repository that unifies them. It has three parts: (i) the missing primitive — a **hierarchical graph container** stated in the language of **sets** and **operators**; (ii) a **formal intermediate representation** — a typed, declarative spec language compiled to a calculus of registered operators — that turns intent into a runnable simulation and that a person or a language model can drive; and (iii) the **experiment** — ten distinct simulations and an emergent ant-trail model, all produced from the same code by changing only the simulation file — with the lessons that follow for how to construct the repository.

Part I — The abstraction

1 Two entities: typed sets and fields

The container is built from two kinds of object: **typed sets** and **fields**. A typed set S_α is a discrete collection of entities of one biological *sort*: membrane particles, cytoplasmic particles, nuclei, metabolites, cells, populations, or organisms (Fig. 1). Each sort has its own state schema and its own admissible operators.

Typed sets are connected by **parent maps**

$$\pi_{\alpha\to\beta} : S_\alpha \longrightarrow S_\beta,$$

which assign each entity of sort α to the entity of sort β that contains it. The *fibre* of such a map over an entity $b \in S_\beta$ is its preimage — everything that maps to b:

$$\pi^{-1}_{\alpha\to\beta}(b) = \{\, x \in S_\alpha : \pi_{\alpha\to\beta}(x) = b \,\}.$$

The fibre of $\pi_{\text{particle}\to\text{cell}}$ over a cell c is thus the set of particles inside c. A cell is not a single such fibre, however, but a *bundle of fibres*: membrane particles, cytoplasmic particles, nuclei, metabolites, and other child sets all map to the same parent cell, and the cell is the union of those fibres. The next section makes this containment graph precise.

A **field** $f : \Omega \to \mathbb{R}^q$ is a continuous quantity defined on its own grid — morphogen concentration, pressure, substrate stiffness, or extracellular chemical concentration. Fields are *not* part of the containment graph: they do not contain cells, and cells do not contain fields. Instead, each field is coupled to one or more typed sets through *exchange* operators — secretion, sensing, particle–grid transfer, or field sampling (Section 3).

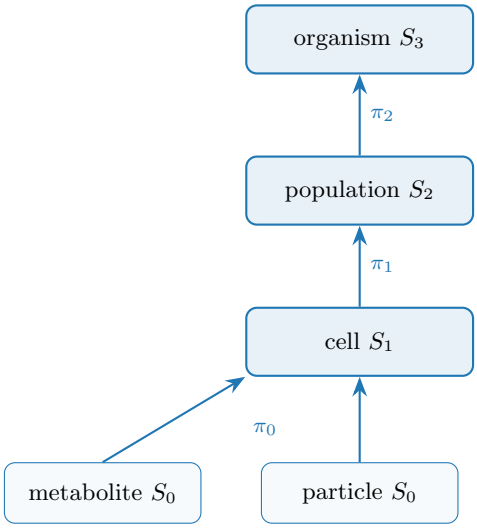


Figure 1: Containment across scales: each parent map π sends an entity to the one that contains it. A cell is a *bundle* of fibres — here its particles and metabolites; in general also its membrane, nucleus, and other child sets (Section 2).

2 A typed containment graph

Section 1 named entities by their sort and joined them with parent maps. Two ideas make that precise, and they are what keep the code free of branching: the sets form a *family* indexed by sort, and the parent maps form a *graph* rather than a chain.

A family of typed sets. Rather than one set S_0 with a type function $\tau : S_0 \to \{\text{membrane}, \text{nucleus}, \dots\}$, we take a family

$$S = \{ S_\alpha \}_{\alpha \in A},$$

one set per *sort*, each carrying its own state schema, operators, and fields. The two are equivalent as sets — the fibres $\tau^{-1}(\alpha)$ *are* the S_α — but the family is the computationally honest presentation: a single set with a type label forces every operator to branch on τ at run time (the spaghetti), whereas a family lets each operator declare *one* sort statically (`at: nucleus`) and never branch. This is many-sorted set theory, the typed discipline an ECS or a relational database already uses.

Joined by containment maps. The sorts are joined by parent maps, declared by naming the parent,

$$\pi_{\alpha \to \beta} : S_\alpha \longrightarrow S_\beta,$$

assigning every entity of sort α to a parent of sort β. A parent may have many children, so a cell is not one fibre but a **bundle of fibres**,

$$\text{Cell}(c) = \left\{ \pi^{-1}_{\alpha \to \text{cell}}(c) \ : \ \alpha \in \text{children(cell)} \right\}.$$

The container is therefore a *typed containment graph* — a **DAG** (directed acyclic graph: the sorts are nodes, the parent maps are directed edges, and containment never cycles), shown in Fig. 2; a DAG rather than a tree because a parent has *many* children. Categorically it is a diagram of sets — a functor from the containment graph (the shape) into **Set** — and the old $S_0 \to S_1 \to S_2$ was just the chain-shaped case. Nothing leaves set theory; the *shape* generalizes from a ladder to a graph. The shift from "level k contains level $k-1$" to "sort A is contained by sort B" is what keeps the code clean: the engine needs only `Level.parent` (the index map) and `Level.parent_name` (the edge); new biological structure is a new set and a parent map, never an engine change.

A new set, or a new type? The type function τ does not disappear; it is demoted. It no longer distinguishes *kinds* — it parameterizes a *subpopulation of one kind*. Two groups that differ in state space or operators are different *sorts* (separate sets); two groups that share both and differ only in parameter values are *types within one set* (the τ colouring, our `types:` block). Table 1 is the rule, and it is the structural form of the category discipline of Section 8: a new *kind* is a new set, a parametric *variant* is a new type.

	Different sorts (typed sets S_α)	**Types within one sort** (the τ colouring)
example	membrane vs. nucleus vs. metabolite	soft cell vs. stiff cell
differ in	state space + operators	only parameter values
in the spec	separate `sets:` with `parent:`	a `types:` block inside one set
dispatch	by name (`at: nucleus`)	per-row param lookup $p[\tau(i)]$

Table 1: When to declare a new set versus a new type: a different *kind* (state space or dynamics) $\Rightarrow$ a new sort; a parametric *variant* $\Rightarrow$ a type within a set.

Keeping operators typed is the payoff: `membrane_tension` acts on `membrane_particle`, `metabolism` on `metabolite`, `gene_regulation` on `nucleus`, and Exchange operators connect compartments explicitly (nucleus $\leftrightarrow$ cytoplasm, membrane $\leftrightarrow$ extracellular field) — never one mega-operator branching on a role flag.

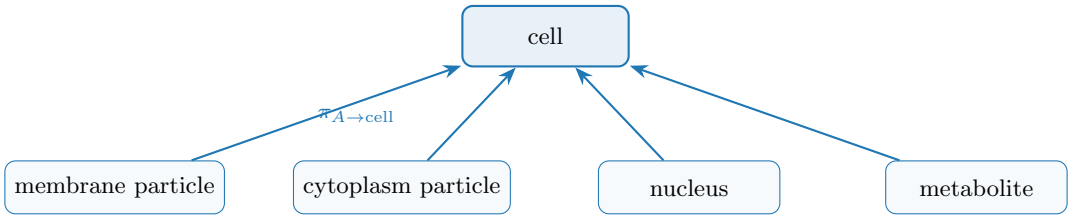


Figure 2: A typed containment graph: one parent set (`cell`) with several child sets, each its own type with its own parent map $\pi_{A\to\text{cell}}$. The cell entity is the bundle of these fibres — not a bag of one kind of particle. A new compartment is a new row in the spec (Listing 1), not an engine edit.

Listing 1: A cell as a bundle of child sets: each compartment is its own typed set with a parent map.

```
sets:
  cell:               {n: 100}
  membrane_particle:  {parent: cell, per_parent: 64,  role: membrane}
  cytoplasm_particle: {parent: cell, per_parent: 200, role: cytoplasm}
  nucleus:            {parent: cell, per_parent: 1,   role: nucleus}
```

```
  metabolite:        {parent: cell, per_parent: 50,  role: metabolite}
```

3 Operators

A GNN is an **operator** $\mathcal{O}$ dispatched by the relation it acts on. An operator changes exactly one of the three things the container holds (Fig. 3): the **state** on the nodes, the **relations** between them, or the **entities** themselves. **Four dynamics operators** move *state* along a fixed structure, one per relation the container admits, each returning a time-derivative contribution. **Rewire** changes the *relations* — the edge set E, which nodes interact. **Divide** and **Die** change the *entities* — the node set $|S_k|$, which nodes exist. Those last three return no state delta. The four dynamics operators:

Lateral	$\mathcal{O}_E,\ E \subseteq S_k \times S_k$	(ODE interaction + graph Laplacian)
Aggregate $\uparrow$	$\sum_\pi : S_k \to S_{k+1}$	(reduction over fibres of π)
Broadcast $\downarrow$	$\pi^* : S_{k+1} \to S_k$	(lift along π)
Exchange	$\mathcal{S}, \mathcal{G} : S_k \leftrightarrow F$	(scatter / gather; bipartite)

Two pieces of notation: $S_k \times S_k$ is the set of all node *pairs*, so an edge set $E \subseteq S_k \times S_k$ is simply a graph on S_k — which nodes interact (e.g. neighbours within a cutoff radius, or a fixed connectome). And $\mathcal{S}, \mathcal{G}$ are the two halves of a field coupling: *scatter* $\mathcal{S} : S_k \to F$ writes object state into the field, while *gather* $\mathcal{G} : F \to S_k$ reads it back. Each operator in turn:

Lateral $\mathcal{O}_E$. Message passing *within* one scale along the edges E. It carries the within-scale physics: pairwise interactions (forces, adhesion, signalling) plus a graph Laplacian that diffuses or smooths quantities over the edges. Examples: particle–particle elasticity, cell–cell flocking or adhesion, neuron–neuron signal propagation.

Aggregate $\sum_\pi$ (up). Pools each parent's fibre into one quantity — a cell's position is the mean of its particles, its metabolic state the sum over its molecules. This is how fine-scale dynamics are summarised into, and so move, the coarser object above.

Broadcast π^* (down). The adjoint of Aggregate: it copies a parent's state to every child in its fibre, so a decision taken at the cell level (a body force, a polarity, a signalling output) is applied identically to all of that cell's particles. Aggregate and Broadcast are a matched up/down pair on the same map π.

Exchange $\mathcal{S}, \mathcal{G}$. Couples a set to a field: scatter $\mathcal{S}$ deposits/secretes object state into the field, gather $\mathcal{G}$ samples/senses the field back onto the objects. The relation is bipartite (objects $\leftrightarrow$ grid); it is exactly the MPM particle–grid transfer (P2G/G2P) and chemotactic secretion/sensing.

Rewire $\mathcal{R}$ (relations). Rewire changes the *relation* $E \subseteq S_k \times S_k$ on which the lateral operators act — *which* nodes are neighbours — and returns no state delta. A *geometric* relation is rebuilt every tick from the live configuration (a **radius graph**: all pairs within a cutoff; or a k-nearest-neighbour or Delaunay graph), so the interaction graph tracks motion, growth, and division; a *fixed* relation (a connectome) is built once and never rewired. Rewire is of a different nature from the

other five: it neither moves state nor changes who exists — it maintains the *scaffold* the dynamics run on. Separating *who* interacts (Rewire) from *how* they interact (Lateral) is also what turns a dense $O(N^2)$ force law into $O(|E|)$ message passing and lets several lateral operators share one graph.

Divide and Die (entities). The dynamics operators move *state* and Rewire moves *relations*, both while the node set stays fixed; living matter also rewrites its own membership. **Divide** $\mathcal{B} : x \mapsto \{x', x''\}$ (proliferation) and **Die** $\mathcal{D} : x \mapsto \varnothing$ (apoptosis / efflux) are the only operators that change the *entities* themselves — the cardinality $|S_k|$. For a *living* system they are as primordial as the four dynamics operators: a tissue is defined as much by the cells it makes and loses as by how they move. Everything else assumes the cells it acts on already exist.

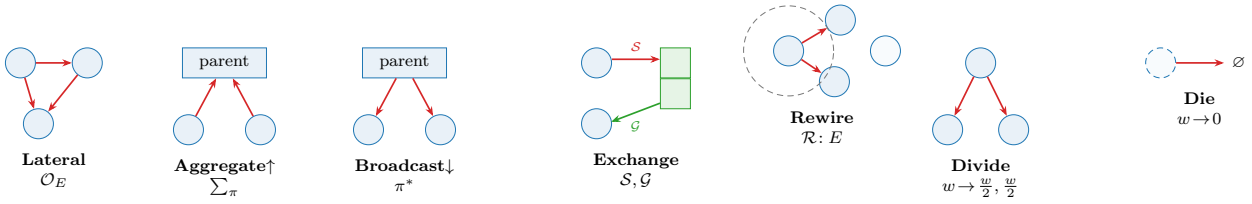


Figure 3: The operators, grouped by what they change. The first four are *dynamics*: they move *state* along a fixed structure — *Lateral* within a set; *Aggregate*/*Broadcast* across containment π; *Exchange* to/from a field (P2G/G2P and the metabolite–reaction graph are both Exchange). The last three return no state delta: *Rewire* changes the *relations* — the edge set E, which nodes interact (a radius graph rebuilt each tick, dashed cutoff); *Divide* and *Die* change the *entities* — the node set $|S_k|$, cell division and death on a fixed buffer with per-element occupancy w.

Differentiating division and death. Changing $|S_k|$ is the one move that resists automatic differentiation: it alters a tensor's shape, and "does this cell divide?" is a discrete event with no useful gradient. We keep the framework differentiable with a **continuous-occupancy** formulation. Rather than resize, we hold a fixed buffer of $N_{\max}$ slots and give every element a *mass* $w_i \in [0, 1]$: `Die` decays $w_i \to 0$ (the slot goes dormant, not deleted); `Divide` activates a dormant slot, copies the mother's state with a small perturbation, and splits the mass $w \to (\frac{w}{2}, \frac{w}{2})$. *Every operator then honours* w: Aggregate becomes a *mass-weighted* reduction $\left(\sum_i w_i x_i\right) / \sum_i w_i$, Lateral weights each message by w_j, and so on — so the boolean selector mask of Section 7 is just the $\{0, 1\}$ special case of w. The trajectory between events is now smooth in w, so gradients flow on a fixed-shape tensor (and `torch.compile` still applies). The one genuinely non-differentiable part — the integer decision of *when* to divide or die — is handled separately: a straight-through or Gumbel estimator to learn a division *policy*, a score-function (REINFORCE) gradient for an expected objective with respect to division/death *rates*, or black-box search (UCB/CEM) when the objective is itself discrete (final count, fraction escaped). This is the *use both* recipe of Part III applied to structure: gradients for the continuous interior, black-box for the discrete choices.

4 Dynamics: a schedule

A simulation is a multi-rate composition of operators, declared as a `Schedule`. In math terms, one tick is the composition $\Phi = \mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$, and the dynamics is its iteration $x_{t+1} = \Phi(x_t)$ — a discrete flow. *Multi-rate* means the operators need not all fire on every tick: a slow process (diffusion, cell

division) can be scheduled to act once every k ticks, while a fast one (mechanics) runs every tick, so a single `Schedule` interleaves dynamics that evolve on different time scales. The `Schedule` is just the code-level encoding of that composition. Each operator is *pure*: instead of editing the state in place, it only *returns its contribution* — a delta Δ_i (a time-derivative term) for the sets it touches. The schedule sums the deltas acting on each set and integrates once per tick.

An operator declares the *order* of its delta, since a time-derivative can be of position or of velocity. A *first-order* (overdamped) operator returns a velocity and integrates as $x_{t+1} = x_t + \Delta t \sum_i \Delta_i$; a *second-order* (inertial) operator returns an acceleration and integrates as $v_{t+1} = v_t + \Delta t \sum_i \Delta_i,\ x_{t+1} = x_t + \Delta t\, v_{t+1}$. The order is a fixed property of each operator (`PREDICTION`): attraction–repulsion is first order, boids flocking is second; all operators acting on one set must agree, so the set integrates as a single order. Friction in the inertial case is itself an operator (a `drag` delta $-\kappa v$), not a special term in the integrator.

Two consequences follow: several operators may act on the same set in one tick (their deltas simply add — none overwrites another), and because nothing is mutated in place, gradients flow through the sums and the integrator, so the whole rollout stays differentiable. The LLM searches over schedules and operator parameterizations — one well-defined action space instead of per-domain code.

5 Glossary

Concept	**Set/operator term**	**Symbol**	**Code**
collection of like nodes	set (a sort)	S_α	`Level`
a single node	element	$x \in S_k$	row of `Level.state`
learnable per-node code	embedding	a_i	`Level.embedding`
containment	parent map (by name)	$\pi_{A\to B} : S_A \to S_B$	`.parent`, `.parent_name`
within-set links	relation	$E \subseteq S_k \times S_k$	`Level.edge_index`
continuum quantity	field	$f : \Omega \to \mathbb{R}^c$	`Field`
dynamics (GNN)	operator: ODE+Laplacian	$\mathcal{O}$	`Operator`
within-set op	lateral	$\mathcal{O}_E$	`Lateral`
up op	reduction over fibres	$\sum_\pi$	`Aggregate`
down op	lift	π^*	`Broadcast`
set↔field op	transfer	$\mathcal{S}, \mathcal{G}$	`Exchange`
rebuild the relation	relations: rewire	$\mathcal{R} : \to E$	`Rewire`
make an element	entities: division	$\mathcal{B} : x \mapsto \{x', x''\}$	`Divide`
remove an element	entities: death	$\mathcal{D} : x \mapsto \varnothing$	`Die`
per-element occupancy	occupancy	$w_i \in [0, 1]$	`Level.occ`
the container	hierarchy	$(S_\bullet, F_\bullet, \pi_\bullet)$	`Hierarchy`
the model	composition of ops	$\mathcal{O}_n \circ \cdots \circ \mathcal{O}_1$	`Schedule`

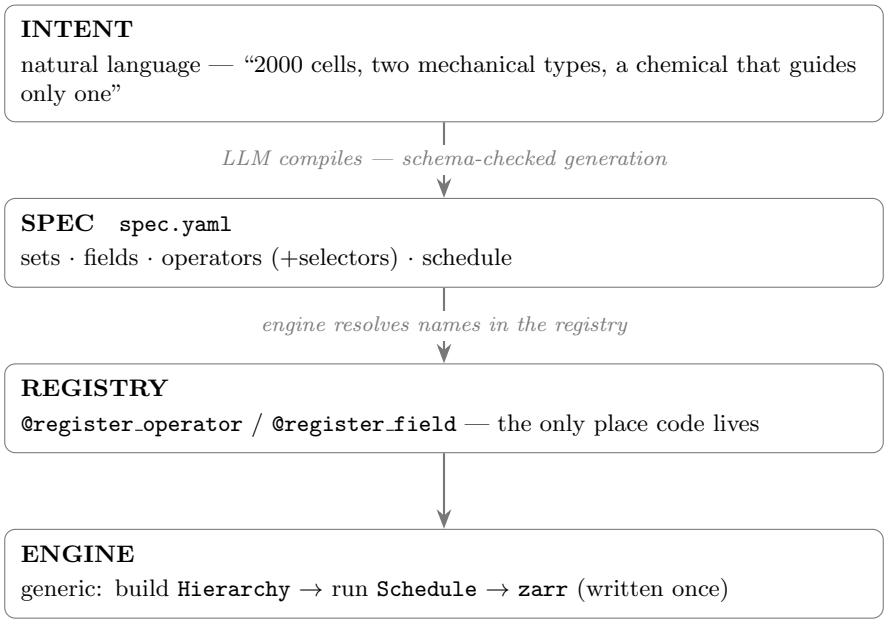
Part II — A formal intermediate representation

6 Two layers and the contract between them

The contribution of this part is not that a language model can drive the framework — it is that there is a **formal intermediate representation** for anything to drive at all. Between a person's *intent* and a running *simulation* we interpose one typed, declarative artifact, the **spec**, and the system becomes a compiler stack:

$$\text{Intent} \longrightarrow \text{Language} \longrightarrow \text{Calculus} \longrightarrow \text{Simulation}.$$

Intent compiles to the **language** (the spec, `spec.yaml`); the spec names primitives in a **calculus** of operators (the registry); the engine **interprets** that calculus into a simulation. In one line: the spec is the *source*, the registered operators are the *primitives*, and the engine is the *interpreter*. A language model is one front end on the first arrow — useful, but replaceable; the durable object is the representation in the middle. Every property claimed below is a property of *it*, not of the model that writes it.



The **spec is the contract**. The LLM owns everything above it (intent → spec); the code owns everything below (operators). They never touch directly. This makes the system both LLM-drivable and human-auditable: the spec is small, typed, and reviewable, unlike the 500-line flat `config.py` it replaces.

Being a small, typed, declarative file gives the contract the properties one wants of an interface between an unreliable generator and exacting code:

- **Verifiable.** The spec is validated against the schema *before* anything runs: an unregistered operator, a missing required property, or a malformed selector is rejected up front, so a spec that loads is guaranteed runnable (the missing-operator and missing-property cases below).

Beyond well-formedness, each simulation also ships an *intent* check that measures whether the requested behaviour actually occurred — "it ran" is not "it worked".

- **Reproducible.** A run is bit-deterministic given (spec, seed) — the engine forces deterministic kernels — so the same spec yields the same trajectory, and a fresh seed draws a new realization of the same dynamics.
- **Saveable.** The spec is ∼20 lines, so it version-controls, diffs, and archives trivially; together with the seed it *is* the complete record. We archive the recipe, not the dish: store spec + seed (plus metrics and a thumbnail) and regenerate the trajectory on demand (`archive.py`, `GALLERY.md`).
- **Composable.** A variant is a base spec plus a handful of overrides, which makes ablations and the optimizer's parameter sweeps cheap.
- **Extensible.** New behaviour is one new registered operator; the operator vocabulary grows monotonically, and every earlier spec keeps working.

7 The simulation language

The contract above is, in effect, a small **domain-specific language** in declarative-data form. Like any language it has *words* (vocabulary), *grammar* (how they combine — here a typed schema, which *is* the language definition: required keys, type fractions summing to one, every operator registered, every schedule token resolving), and *semantics* (what running them means: the engine builds the `Hierarchy` and iterates the schedule, $x_{t+1} = \Phi(x_t)$). Table 2 is the complete vocabulary; every construct in it appears in the example of Listing 2.

Structurally this is an **entity–component–system** (ECS) architecture: sets are entities, their state buffers are components, operators are systems, and the schedule is the ECS system pipeline. We adopt that framing deliberately — it is the cleanest mental model and matches the engine one-to-one. Each entity kind declares its component layout once in the registry (`@register_entity`): a typed *state schema* naming which state columns are position, velocity, and so on, plus *render* hints (what to colour by, whether to draw orientation). The engine, the operators, and the visualizer all read column semantics from this one declaration rather than hard-coding offsets like `state[:, :2]` — so a new entity is drawn and integrated correctly for free.

Most of the spec is plain typed key–value; the only genuine sub-grammars are the last two blocks of Table 2 — *selectors* and the *schedule*. Both are deliberately minimal but extend naturally: selectors to conjunctions and to geometric or temporal predicates (`cell[type=soft & loaded=0]`, `cell[x<0.5]`, `cell[age>10]`), and the schedule to explicit per-operator rates and nested loops.

We keep the language as *declarative data* (YAML/JSON) rather than a bespoke syntax, so an LLM can emit it under schema-constrained decoding and a human can diff, review, and archive it; the grammar is pinned by a typed schema (Pydantic or JSON-Schema), and composition, overrides, and parameter sweeps (variants, ablations, the optimizer) are delegated to a config layer such as Hydra rather than reinvented. Prior art worth borrowing: ECS engines (the scheduling model), SBML/Antimony (reaction–species vocabulary), NeuroML and MorpheusML/PhysiCell (multicellular specs), UFL (PDE operators), and NetLogo/Mesa (agent rules).

A complete spec is a single `spec.yaml`. Listing 2 assembles the vocabulary of Table 2 into one: two cell `types`, `particles` contained in each `cell`, one chemical `field`, and a `schedule`. Soft cells

`secrete` the chemical and `sense` (climb) it, so they self-aggregate, while stiff cells — selected out by `cell[type=soft]` — do not.

Listing 2: A complete simulation: soft cells secrete and follow a chemical and self-aggregate.

```
general: {name: aggregate, seed: 0, n_frames: 250, dt: 0.05, boundary: periodic}
sets:
  cell:
    n: 100
    types: {soft: {fraction: 0.5, youngs: 12}, stiff: {fraction: 0.5, youngs: 300}}
  particle: {parent: cell, per_parent: 100, radius: 0.02}   # containment
fields:
  chemical: {frame: grid, res: 96, diffusion: 0.1, decay: 0.05, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 15.0, rot: 0.4}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 800, drag: 4}
  - {op: secrete, at: "cell[type=soft]", to: chemical, rate: 1.0}    # only pop 1 makes it
  - {op: sense,  at: "cell[type=soft]", from: chemical, gain: 150.0} # only pop 1 follows it
schedule: [aggregate, [motility], secrete, chemical.diffuse, sense, mpm]
```

The validator (prototype: `scenario_schema.py`) guarantees that a file which loads is runnable: every `op` exists; every `at`/`to`/`from` references a declared set/field; type `fraction`s sum to one; every schedule token resolves. (One trap: never use a YAML 1.1 boolean key such as `on`/`off` — we use `at`.)

8 Categorical discipline: what belongs where

The same structure that makes the spec powerful makes it fragile: it has a few *orthogonal categories*, and every key belongs to exactly one. The schema catches a malformed spec, but it cannot catch a *well-formed* spec that files a quantity under the wrong category — a force constant placed on a set, a domain property placed on an operator, an integration knob placed on a particle. These are **category errors**, and for anything editing the representation — a language model most of all — they are the dominant failure mode. The type system cannot state this discipline, so we state it here, as a rule.

The categories. Every part of a spec answers exactly one question, and that question fixes its home (Table 3).

The litmus test. When unsure where a quantity goes, ask whether it is a property of the *thing* or of the *process* acting on it. A property of the thing — its identity, count, composition, the meaning of its state columns, its initial value — belongs to the set/entity. A property of the process — a rate, a coefficient, a force law, the order of the time-derivative it produces — belongs to the operator. A property of the shared space belongs to the world; a property of timing, to the schedule. *Thing → set; process → operator.*

Worked category errors. The tempting move is almost always to attach a *process* parameter to the *thing* it acts on. Table 4 collects the ones we actually hit while building the framework.

Invariants (hold the line).

- An operator never declares what a node *is* (count, type, composition, initial state); it only *reads* those.

- A set never declares how state *changes* — no rates, force constants, or integration order on a set; those are operators or the schedule.
- The world appears once, never per-operator: every spatial operator reads one boundary and one domain.
- Integration is the schedule's job, not an operator's; the only thing an operator says about integration is the *order* of its delta (`PREDICTION`).
- A per-type property is owned by the type (`types:`) and read by whatever operator needs it (declared in `REQUIRES_TYPE_PROPS`), resolved along the containment chain.
- Style never changes dynamics: `plotting:` is read only by the visualizer.

The discard rule. When a quantity does not obviously fit, do *not* invent a new top-level key. Place it in the category whose question it answers; and if no operator or entity reads it, the validator's *"read by no operator"* warning (Section 11) is the tripwire that catches the misfiling. New *behaviour* is a new operator, never a set field; a new *kind of node* is a new entity, never an operator parameter. Hold this line and the action space stays the single, well-defined one a search — or a language model — can explore.

9 Operator catalog

Operators are the system's vocabulary; it grows monotonically and every past spec keeps working. The catalog, grouped by what each operator changes (Section 3); *1st/2nd* marks the integration order of the dynamics operators:

op	kind	acts on	behaviour
Dynamics — move state			
`attraction_repulsion`	lateral *1st*	particle	per-type difference-of-Gaussians interaction law
`boids`	lateral *2nd*	particle/cell	flocking (cohesion/align/separate)
`drag`	lateral *2nd*	any	viscous friction $-\kappa v$
`motility`	lateral	cell	active self-propulsion (wandering heading)
`adhesion`	lateral	cell	type-specific cohesion (+ `cross`<0 repels → sorting)
`mpm`	exchange	particle	MLS-MPM mechanics; soft/stiff via `youngs`
`secrete`	exchange	cell→field	deposit into a field (or consume, rate<0)
`sense`	exchange	cell←field	chemotactic accel up-gradient
`graze`	exchange	cell←field	consume a field (capped at stock); harvest objective
`trail`	exchange	cell	Physarum 3-sensor pheromone steering (ants)
Relations — change the edge set E			
`radius_graph`	rewire	particle	neighbour graph: pairs within a cutoff, rebuilt each tick
`ring`	rewire	particle	closed loop over a role's particles (a membrane)
Entities — change the node set $\|S\|$			
`divide`	structural	cell	cell division: split occupancy w into a new slot
`die`	structural	cell	cell death/efflux: occupancy $w \to 0$
Builtins			
`aggregate`	builtin	—	parent state = (mass-weighted) mean of children (↑)
`<field>.diffuse`	builtin	—	field self-update (Laplacian + decay)

10 Natural language → simulation: the repeatable mapping

To make the LLM's compilation *repeatable* (same request → same structure), it follows fixed rules; the most useful:

1. "N cells/agents" → a set with `n: N`.
2. "made of K X per cell" → a contained set with `parent`/`per_parent` (two levels).
3. "two types differing by P" → `types:` with P as a per-type property.
4. "mechanical/elastic/rigid" → particles + `mpm`; map stiffness to `youngs`.
5. "moves by boids / flock" → `boids`; "wanders / motile" → `motility`.
6. "creates/secretes a chemical" → a `field` + `secrete`; "follows / chemotaxis" → `sense`; "ant trails" → `trail`.
7. "particles attract/repel, interact by type" → a `radius_graph` (the relation) followed by a lateral interaction op (e.g. `attraction_repulsion`); the per-type law parameters go in `types:`, read by the operator.

8. **"only population X does Y"** → the selector `at: cell[type=X]`. This is the highest-leverage primitive: it scopes behaviour to a subpopulation and lets two behaviours coexist.

Throughout, the mapping respects the categories of Section 8: a *behaviour* becomes an operator, a *kind of node* a type, the *space* the `general:` block — never the reverse.

11 When the present code is not enough

The gatekeeper fails loudly and points at the gap — never with a crash deep in a substep — in two forms.

A missing operator. A request needing a behaviour no operator provides (say reaction–diffusion) is rejected at load:

```
operator 'reaction_diffusion' not in registry.
Available: ['attraction_repulsion','boids','drag','radius_graph','divide','die', ...]
```

The fix is **one new `Operator` subclass** — never an engine change: write `forward(self, H, mask) -> set: delta` (or mutate a field/relation and return {}); decorate `@register_operator`; declare its needs; reference it in the spec. We exercised this live during the experiment: adding `adhesion` unlocked cell *sorting* and `trail` unlocked ant *trail-following* — each ∼20 lines, the engine and every other spec untouched.

A missing property. Operators *declare* what they need (`REQUIRES_PARAMS`, `REQUIRES_TYPE_PROPS`); the validator checks before running, resolving a property along the containment chain (`mpm` acts on particles but `youngs` lives on the parent cell's types):

```
# missing required property -> hard error, before any computation:
operator 'mpm' requires property 'youngs' on every type of 'cell'; missing on type 'soft'.
# property no operator reads -> warning (typo guard + category-error tripwire):
[warn] property 'viscosity' on cell.soft is read by no operator (known: ['fraction','youngs'])
```

Design rule: capabilities are declared, not discovered at a crash. The operator that consumes a property is the single place that declares the requirement, turning a would-be `AttributeError` thousands of substeps deep into a one-line message up front — and that "read by no operator" warning is exactly the tripwire for the category errors of Section 8.

Part III — The experiment

12 A spread of simulations from one registry

Once the vocabulary existed, qualitatively different behaviours were *just different specs* over the same code — breadth comes from the intermediate representation, not from new code. Each simulation is 100–600 cells of ∼100 MLS-MPM particles (the two cell types in two colours over the shaded chemical field); eight are shown in Fig. 4:

- **aggregate** — soft cells secrete a chemical and climb it, clumping together;
- **disperse** — soft cells flee their own chemical (negative gain), spreading apart;

- **swarm** — strong boids make the cells flock and move as one;
- **wander** — active motility alone: a diffusive gas of cells;
- **sort** — type-specific adhesion segregates the two populations;
- **chase** — a fast-decaying field makes moving hotspots the cells chase;
- **crystal** — pure repulsion spaces the cells into a lattice;
- **collapse** — low diffusion with strong chemotaxis pulls soft cells into one tight cluster.

Across all eight, only the operator choices and their parameters differ; the engine, the operators, and the schedule grammar are identical.

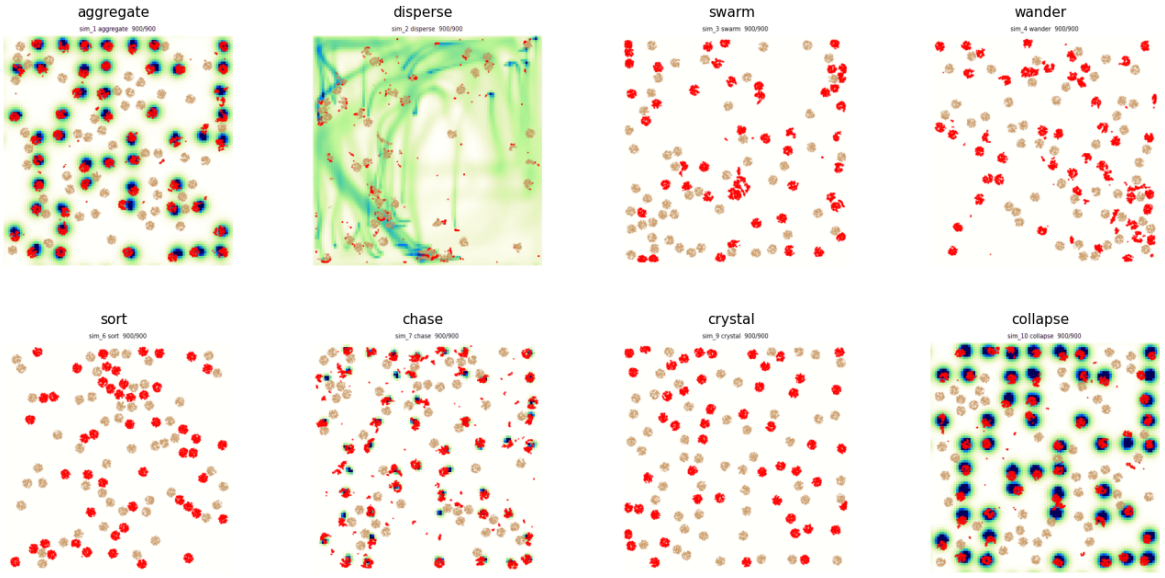


Figure 4: End states of the eight simulations described above, all generated from the same operator registry by changing only the spec.

13 Emergent ant trails

Adding the `trail` operator (a Physarum/Jones three-sensor rule: each ant samples pheromone ahead-left / ahead / ahead-right and turns toward the strongest, while `motility` drives it forward and `secrete` lays pheromone) yields self-reinforcing trails that grow, are followed, and coarsen over long runs (Fig. 5). Listing 3 is the full simulation.

Listing 3: The ant simulation: red cells lay, follow, and reinforce pheromone trails.

```
name: ant
seed: 0
n_frames: 1500
sets:
  cell:
    n: 150
    types: {soft: {fraction: 0.6, youngs: 60}, stiff: {fraction: 0.4, youngs: 300}}
  particle: {parent: cell, per_parent: 60, radius: 0.012}
```

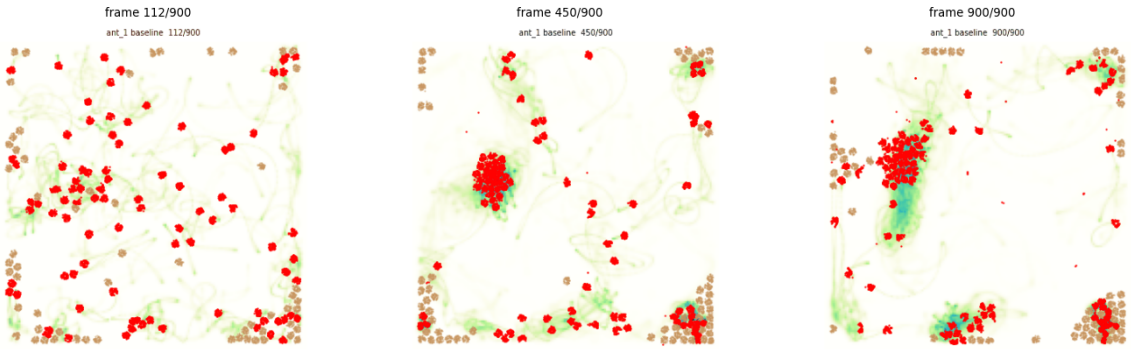


Figure 5: Ant trail-following over a long run (three frames, early to late). A diffuse network of pheromone trails (shaded) forms, reinforces, and *coarsens into a few dominant trails* with ants marching along them (right) — classic stigmergy.

```
fields:
  pheromone: {frame: grid, res: 160, diffusion: 0.02, decay: 0.04, couples_to: cell}
operators:
  - {op: motility, at: cell, speed: 300.0, rot: 0.06}            # persistent forward motion
  - {op: trail,  at: "cell[type=soft]", from: pheromone, turn: 0.4, sensor_dist: 0.04, sensor_angle: 0.5}
  - {op: secrete, at: "cell[type=soft]", to: pheromone, rate: 2.0}
  - {op: mpm, at: particle, n_grid: 128, substeps: 8, a_max: 1200, drag: 0.6}
schedule: [aggregate, trail, motility, secrete, pheromone.diffuse, mpm]
```

14 Cells in a maze: foraging and optimization

A harder simulation exercises more of the language: two rigid cell populations start at *home* (bottom-left), navigate a walled *maze* to *food* (top-right), pick food up, and carry it back. It adds three constructs — `obstacles`, a per-cell `loaded` state, and state-gated selectors (`cell[loaded=0]` seeks food, `cell[loaded=1]` returns home) — and answers the design question *how does one model an obstacle*: a wall is not a pairwise force but a **static occupancy mask reused as a boundary condition** by machinery already present. The same mask (i) zeroes the MPM grid velocity inside walls, so rigid cells cannot enter, and (ii) imposes no-flux on field diffusion, so a chemical *routes around* the walls. Two such fields — one sourced at food, one at home — become navigation gradients that solve the maze; unloaded and loaded cells climb opposite ones.

Because the whole simulation is differentiable and the spec is tiny, the colony can be *optimized*: search the behavioural parameters to deliver food as fast as possible. The objective — units delivered — comes from a discrete pick-up/drop-off and is therefore non-differentiable, so we use a black-box **UCB** search over eight levers (motility speed and rotation, sense gain, MPM drag and force cap, cell stiffness and size, colony size). Over a few dozen simulations it improves delivery roughly five-fold. This echoes the differentiable-simulator design literature, where a *smooth* objective is optimized by gradients through the rollout; the general recipe is **both** — UCB or CEM for the discrete and structural choices, and gradient-through-rollout on a smooth surrogate for the continuous interior.

The optimizer keeps a log of every accepted improvement (the auto-generated `WINNERS.md`, Table 5), which is itself the finding: it shows *what makes a good forager*. Two parameters move monotonically

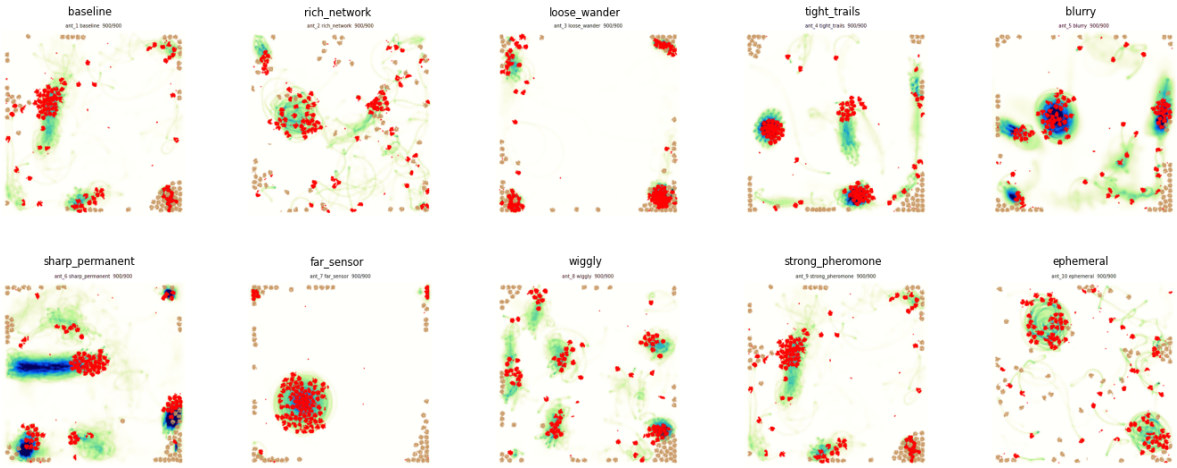


Figure 6: Ten ant variants, each the same `trail+motility+secrete` code with a few parameters changed (turn rate, sensor reach/angle, pheromone diffusion/decay, deposit rate, rotational noise). The parameter axes span the trail-formation regime — from loose wandering to tight reinforced networks.

to their bounds — the colony grows to its cap ($n \approx 120$: more foragers deliver more food) and the cells soften to the floor (youngs $\to 20$: soft, deformable cells squeeze through the maze far better than the rigid ones we started from). Chemotaxis stays strong throughout (gain $\sim$ 340–400) and the cell radius sits at its minimum (smaller cells clear the gaps). The remaining knobs — motility speed and rotation, drag, force cap — are traded off rather than maximized: the best design (`winner_7`) is a balanced, moderate-speed, moderate-drag colony, not any extreme. UCB explores widely (the parameter columns are non-monotonic) while the best-so-far food rises monotonically by construction. The headline is that the search rediscovered, with nothing hand-coded, that a *large swarm of small, soft, strongly-chemotactic cells* forages a maze best.

The difference is visible in the trails themselves (Fig. 7): the initial colony barely establishes a route, whereas the optimized one lays a strong chemoattractant highway that threads both maze gaps from home to food.

15 Races: a longitudinal world, an exit boundary, and more levers

The foraging maze generalizes into a family of *races*: a population starts at the left and must reach an exit at the right as fast as possible. Three additions to the language carry them, none touching the engine's generic core.

A longitudinal world. A declarative `world: W` makes the domain the rectangle $[0, W] \times [0, 1]$ on a grid of *square* cells ($n_x = \text{round}(W\, n_y)$); W=4 gives a long left-to-right track. Obstacles may now be **circles** (`[cx,cy,r]`) as well as rectangles, so a porous field of ~hundreds of round pillars and a real recursive-backtracker maze (genuine corridors and dead ends) are both just obstacle lists, reused as the one MPM/field mask as before.

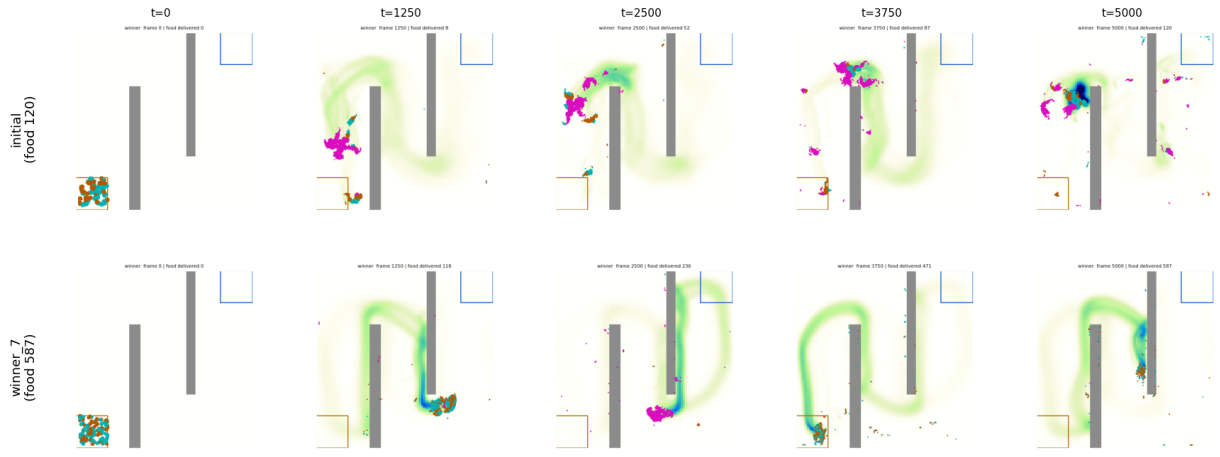


Figure 7: Initial design (top) versus the best found by UCB (bottom, `winner_7`), each a single run shown as five frames from early to late. The optimized colony forms a far more complete and persistent foraging trail.

An exit (efflux) boundary. A cell crossing the finish is handled by an operator: `finish` *freezes* it at the line, while `death` *removes* it (zeroing its particles' mass and parking them off-domain) so it disappears instead of piling up. Both count the crossing in `H.finished` (the race objective) and set a sticky `done` flag; driving operators select `cell[done=0]`, so an exited cell stops being driven or drawn. `death` is the discrete-set counterpart of a field `source`: an open sink, run last in the schedule. Navigation reuses the proven hidden *geodesic* field (no source bands), while only the faint deposited `scent` trail is shown.

More levers, including `boids`. The optimizer (`race_opt.py`, same kNN-UCB as foraging, a winner gif saved whenever the escaped count rises by ten) sweeps the foraging levers *plus the three boids weights* and its radius — `boids.cohesion/align/separate/radius` — twelve in all. This is the point of the operator-parameter view: the search tunes the collective-motion rule (how strongly cells flock and avoid jamming in a tight maze) jointly with body and fleet, not just scalar gains.

16 What the process taught us

1. **The intermediate representation is the whole game.** Ten distinct behaviours and an ant model were all different YAML over the same code.

2. **Selectors are the highest-leverage primitive.** `cell[type=soft]` expresses "only population 1 does X" and lets behaviours coexist; every operator must honour the selector mask.

3. **Verification must check *intent*, not just "it ran".** The hard bugs were physical: unstable diffusion ($dt \cdot D > 0.25$), wrong particle volume, unclamped accelerations blowing up MPM, cell inversion at very low stiffness, GPU non-determinism breaking reproducibility, and — subtlest — the *wrong metric* (net displacement vs. nearest-neighbour clustering).

4. **Honesty about regime.** Self-secreted chemotaxis aggregates only below a density threshold (above it the field is spatially flat — correct physics); the simulation/verify output should

state the regime, not silently show the one setting that worked.

5. **Determinism is a feature you must force** (deterministic CUDA kernels), or "reproduce with a new seed" is meaningless.

6. **Obstacles are boundary conditions, not forces.** A wall is one static mask reused by the MPM grid (interior zero-velocity) and the diffusing fields (no-flux); maze navigation then falls out of the field's gradient, with no special pathfinding code.

7. **Cap speed below the CFL limit.** Strong gradients otherwise drive cells to the grid's stability ceiling, where they move ballistically ("asteroids"); an explicit max-speed clamp plus overdamped drag restores smooth, biological motion.

8. **Discrete objectives need black-box search, differentiable ones admit gradients — use both.** Food count (discrete) optimizes well under UCB; a smooth surrogate opens the gradient-through-rollout path, and the two compose: UCB for structure, gradients for the continuous interior.

9. **Reproducibility needs full-precision provenance.** Archiving rounded parameters made a winner re-simulate to a different score; the exact spec (and seed) must be stored, or the run is not bit-reproducible.

Part IV — Building the repository

17 Proposed methodology

Layering (strict; no leaks across).

- `models/base.py` — entities, operator base, the capability-contract fields.
- `models/registry.py` — the only name→code map (entity / operator / field).
- `models/<domain>/*.py` — operators, one concern per file, each declaring `REQUIRES_*`.
- `schema.py` — the schema + validator (the contract gatekeeper).
- `engine.py` — generic; contains **no** simulation-specific logic.
- `viz.py` — generic over the `Hierarchy/zarr`; swappable backends (2-D now, 3-D later).
- `archive.py` + `simulations/` — the growing, reproducible gallery.

The build loop (how LLM and code co-evolve).

1. user states intent in natural language;
2. LLM compiles → `spec.yaml` (following the mapping rules);
3. validator runs: unknown op → write one operator; missing property → fix spec or add the requirement; else it is runnable;
4. engine runs; **verify** checks intent + repeatability + well-formedness;

5. `archive.py` records spec+seed+metrics+thumb into the gallery;
6. new capability → one new registered operator (+ its `REQUIRES_*`), never an engine edit.

Invariants to hold the line.

- Code lives only behind the registry; the engine never branches on simulation names.
- Every operator honours its selector mask and declares its `REQUIRES_*`.
- Every simulation ships an intent check; "it ran" $\neq$ "it worked".
- Archive the recipe (spec+seed); regenerate the dish.
- When a result holds only in a regime, the spec/verify says so.

This is the contract that lets the LLM operate at the top (intent → simulation, plus occasional single-operator authoring) while the low-level code stays a clean, growing, verifiable registry underneath.

Appendix

A Plexus and Lean

At first glance Plexus and a proof assistant such as Lean appear unrelated. One simulates living systems; the other verifies mathematics. Yet they share a surprisingly similar architecture. Both seek to construct complex objects from a small set of primitive building blocks while preserving correctness. Understanding this correspondence helps clarify what Plexus actually is: not merely a simulator, but a formal system for describing biological models.

A.1 A brief introduction to Lean

Lean is an interactive theorem prover developed to formalize mathematics. A user states a theorem and constructs a proof. Unlike a traditional mathematical proof written for humans, a Lean proof must be expressed in a precise formal language that the Lean kernel can verify mechanically.

For example, the mathematical statement

$$a + b = b + a$$

can be expressed as a theorem in Lean:

```
theorem add_comm (a b : Nat) : a + b = b + a := ...
```

The dots represent a proof. Once supplied, Lean checks every step and certifies that the conclusion follows from the rules of the system.

Lean is built around several layers.

1. **Types**. Every object belongs to a type: natural numbers, real numbers, sets, functions, and so on.

2. **Terms**. Concrete objects inhabit types. The number 3 is a term of type `Nat`. A function is a term of a function type.
3. **Theorems**. A theorem is a proposition that one wishes to prove.
4. **Proof terms**. A proof is itself an object. Under the Curry–Howard correspondence, propositions behave like types and proofs behave like terms inhabiting those types.
5. **The kernel**. A small trusted program checks that a proof is valid.
6. **Tactics**. Convenient automation that helps construct proofs. Tactics can be sophisticated and even unreliable internally because the kernel checks the result afterward.
7. **Mathlib**. A large library of previously formalized mathematics.

The key idea is separation of concerns. The user writes proofs, tactics help construct them, and the kernel verifies them. Trust resides in the kernel rather than in the process that produced the proof.

A.2 The structural correspondence

Plexus exhibits a remarkably similar decomposition. The correspondence is not perfect, but it is useful.

Lean	**Plexus**	**Role**
Types	Sets S_α and Fields F	primitive objects
Terms	Entities (cells, particles, metabolites)	instances of those objects
Functions	Operators	transformations
Proof composition	Schedule	composition of transformations
Type checker	Schema validator	correctness gate
Tactics	LLM compiler	search / generation layer
Theorem	Intent	desired outcome
Proof term	Simulation specification	executable description
Kernel	Engine	trusted interpreter
Mathlib	Operator registry	reusable library of primitives

Several rows deserve explanation.

A typed set in Plexus plays a role similar to a type in Lean. Just as Lean distinguishes natural numbers from matrices or groups, Plexus distinguishes cells, particles, metabolites, nuclei, and fields. Operators declare which kinds of objects they act upon, exactly as functions declare their input types.

Entities are analogous to terms. A particular cell is an element of the set `cell`, just as the number 3 is an element of the type `Nat`.

The operator registry resembles a mathematical library. New functionality is added by registering a new operator rather than modifying the engine itself. Existing simulations continue to work because the vocabulary only grows.

The strongest correspondence is architectural. The simulation specification (`spec.yaml`) is generated by a human or by an LLM. It is not trusted. The schema validator checks that the specification is well formed before execution. Only after passing validation is it handed to the engine. This mirrors the relationship between tactics and the Lean kernel.

A.3 Why the analogy is imperfect

Despite these similarities, Plexus is not a theorem prover.

Lean is fundamentally concerned with *truth*. Given a proposition P, Lean determines whether a proof exists. If the kernel accepts the proof, the theorem is established with mathematical certainty.

Plexus is concerned with *dynamics*. Given an initial state and a collection of operators, Plexus computes

$$x_{t+1} = \Phi(x_t).$$

The result is not a proof but a trajectory.

This difference has important consequences.

First, the Plexus validator checks only *well-formedness*, not correctness. A specification that passes validation may still represent a poor biological model. The analogue of theorem proving in Plexus is not validation but experimental verification.

Second, Lean computations terminate. A proof either checks or it does not. Plexus simulations may be unstable, chaotic, or sensitive to parameters. The engine integrates a dynamical system rather than reducing an expression to normal form.

Third, Plexus is differentiable. Gradients flow through operators, schedules, and rollouts. This allows optimization of model parameters and even biological designs. There is no direct analogue of differentiation in theorem proving.

A.4 What Plexus borrows from theorem provers

The value of the comparison is therefore not that Plexus proves theorems. Rather, it inherits a design philosophy from systems such as Lean.

The first lesson is to separate a symbolic description from its execution. A simulation should be represented explicitly rather than being scattered across code.

The second lesson is to separate a trusted kernel from untrusted generation. The LLM may produce a simulation specification, but correctness of the specification is established by the validator and the engine, not by the model that wrote it.

The third lesson is that a small set of primitives can support a large space of constructions. Lean builds mathematics from types, functions, and proof rules. Plexus builds biological models from sets, fields, operators, and schedules.

A.5 A language and calculus of living systems

Seen from this perspective, Plexus is best understood as a formal language and calculus for living systems.

The language is the simulation specification. It provides a symbolic vocabulary for describing entities, fields, relations, and schedules.

The calculus is the collection of operators acting on those objects. Lateral, Aggregate, Broadcast, Exchange, Rewire, Divide, and Die are the primitive transformations from which larger biological mechanisms are assembled.

The engine plays the role of an interpreter, turning symbolic descriptions into dynamical trajectories.

This is ultimately the deepest connection to Lean. Both systems seek a compact set of primitives from which a much larger space of structures can be constructed. Lean does this for mathematics. Plexus attempts to do it for biological systems.

B Plexus, World Models, and Mechanism Search

The goal of Plexus is not merely to simulate biological systems. It is to provide a language in which biological mechanisms can be expressed, compared, and discovered.

This distinction became apparent during an agentic modeling experiment on *Dictyostelium* aggregation. An agent was tasked with reproducing a real biological morphology from experimental observations. The search successfully explored many parameterizations of chemotaxis, adhesion, relay signaling, saturated sensing, density repulsion, and deformable mechanics. Several mechanisms were falsified, and one family of models transiently reproduced the target morphology before collapsing at longer times.

At first sight this appeared to be a failure of optimization. In retrospect it revealed a more important limitation: the search explored parameters within a mechanism, but not the space of mechanisms themselves.

Human observers immediately proposed alternative explanations:

- long-range attraction between aggregates,
- external radial fields,
- soft-body fusion,
- coarsening dynamics,
- three-dimensional aggregation projected into two dimensions.

None of these hypotheses were explored. The reason was not that they were unlikely, but that they were absent from the search space. The loop performed local refinement around an existing model rather than proposing qualitatively different explanatory worlds.

The lesson is that scientific discovery requires two distinct searches:

$$\text{mechanism search} \quad \gg \quad \text{parameter search}.$$

The first determines *what kind of world* might explain an observation. The second determines *which parameters* within that world are most consistent with the data.

C World Models and Plexus

Modern world models approach this problem from the opposite direction.

A predictive model such as JEPA or a video foundation model learns a latent representation

$$\text{video} \longrightarrow z_t \longrightarrow z_{t+1},$$

where the latent state z_t is optimized for prediction. Such models can be extremely successful at forecasting future observations, but the latent representation is not required to correspond to interpretable biological mechanisms.

Plexus instead begins from explicit objects:

$$(S_\bullet,\, F_\bullet,\, \pi_\bullet)$$

and explicit operators acting on them. Cells, particles, fields, and interactions remain visible throughout the computation.

The difference is therefore not prediction versus simulation. Both can predict.

The difference is the object being searched.

	World Models	**Plexus**
Primitive representation	latent variables	sets, fields, operators
Search space	latent trajectories	mechanisms and parameters
Primary strength	prediction	hypothesis testing
Interpretability	low–moderate	high
Counterfactuals	implicit	explicit
Scientific hypotheses	difficult to extract	native objects

The two approaches should therefore be viewed as complementary rather than competitive. A world model discovers regularities. Plexus evaluates candidate mechanisms capable of generating those regularities.

D A Mechanistic World Model

The Dicty experiment suggests that a successful discovery system requires an intermediate layer between observations and simulations.

Rather than learning latent dynamics directly, the objective is to learn a mapping

$$\text{simulation specification} \longrightarrow \text{mechanistic description}.$$

Every simulation in the Plexus gallery already contains three linked objects:

$$(\text{specification},\ \text{trajectory},\ \text{mechanistic explanation}).$$

For example:

> "Long-range attraction produces coarsening and eventual fusion into a dominant aggregate."

or

> "A reaction–diffusion field generates filamentary transport networks through local activation and long-range inhibition."

These descriptions form a growing knowledge base connecting operators and morphologies.

The objective is therefore not to learn

$$\text{video} \rightarrow \text{future video},$$

but rather

$$\text{video} \rightarrow \text{mechanistic hypotheses.}$$

This representation remains interpretable because its primitives are the same objects used by the simulator itself.

E The Mechanism Search Tree

Once mechanisms are represented explicitly, the search process changes.

Traditional optimization explores a parameter space:

$$\theta_1, \theta_2, \ldots$$

for a fixed model.

Plexus instead organizes discovery as a hierarchical search over mechanisms.

$$\text{Observation} \rightarrow \text{Mechanism Family} \rightarrow \text{Mechanism Composition} \rightarrow \text{Parameter Refinement.}$$

The resulting search tree resembles

```
Explain observed morphology
|
+-- Long-range attraction
|   +-- attraction radius
|   +-- attraction strength
|
+-- Radial field
|   +-- field shape
|   +-- sensing gain
|
+-- Soft-body fusion
|   +-- Young's modulus
|   +-- surface tension
|
+-- Chemotactic relay
    +-- diffusion
    +-- decay
```

A bandit strategy such as UCB can then allocate computation not only between parameter values but also between competing explanatory worlds.

The crucial distinction is that branches correspond to scientific hypotheses, not merely numerical settings.

F Roadmap

The proposed roadmap consists of four stages.

Stage 1: Mechanistic Captioning. Construct a library of simulations spanning the operator vocabulary of Plexus. Each simulation is paired with a structured mechanistic description.

$$\text{spec} \leftrightarrow \text{mechanism} \leftrightarrow \text{video}.$$

Stage 2: Mechanistic Retrieval. Given a new observation, retrieve simulations with similar morphology and collect their associated explanations.

Stage 3: Mechanism Search. Use the retrieved explanations to initialize a mechanism search tree. The agent explores families, compositions, and parameterizations using simulation-guided evaluation.

Stage 4: Mechanistic World Models. Train models that map observations directly to distributions over candidate mechanisms. The world model no longer predicts future pixels. Instead, it predicts which explanations are worth testing.

$$\text{Observation} \rightarrow \text{Mechanism Prior} \rightarrow \text{Plexus Search} \rightarrow \text{Simulation} \rightarrow \text{Falsification}.$$

In this view, Plexus is neither a simulator nor a world model alone. It is a language in which candidate worlds can be written, searched, and falsified.

G Rules for a well-formed operator

An operator is the only place new dynamics enter Plexus, so a few simple rules keep the calculus sound and the engine generic. They are enforced — by the schema validator before a run, and by the engine on the first frame — so a violation fails loudly rather than silently corrupting a trajectory.

1. The integration invariant. No operator writes the engine-integrated state (pos/vel) directly. A change to position or velocity must flow through the operator's *returned delta*, which the engine integrates once per tick. Everything else is mutated in place and the operator returns {}: the relations E (`edge_index`), the entities $|S|$ (`occ` and per-node buffers, structural operators only), the fields F (`grid`), and auxiliary per-node control buffers (e.g. a slime agent's `heading`, which is not integrated state). A dynamics operator that Euler-steps `pos` itself is a category error — exactly the bug an early `advance` contained, fixed by returning a velocity $v_i = s_i(\cos h_i, \sin h_i)$ and letting the engine do $x \leftarrow x + \Delta t\, v$.

2. One operator, one category. The operator's `KIND` must match what it changes: `lateral`/`aggregate`/`broad` move *state* (return a delta); `rewire` changes the *relations* E; `structural` changes the *entities* $|S|$. An operator never does two of these at once.

3. `at:` names the locality; `to:`/`from:` the field. `at:` is the set an operator acts on, or — for a field-internal operator such as `diffuse`/`decay` — the field it reads and writes. A coupling operator names its field in `to:` (scatter, set→field) or `from:` (gather, field→set). So `diffuse` reads `at: chemical`, never the placeholder `at: cell`.

4. `PREDICTION` matches the derivative order. A force-emitting operator declares whether its delta is a velocity (`first_derivative`, overdamped) or an acceleration (`second_derivative`, inertial); all operators acting on one set must agree, since a set integrates as a single order. Operators that emit no force (`rewire`, `structural`, field writes) set `PREDICTION = None`.

5. Style stays in `plotting:`. Colours, colormap, point size, gamma, and background are render *style*; they live in the spec's `plotting:` block, read only by the plotter. They are never properties of a set or an operator (which carry only what the dynamics read).

6. Dispatch tags are class attributes; values are spec. `KIND`, `LEVEL`, and `PREDICTION` are fixed dispatch tags stamped on the class. Tunable numbers (rates, gains, coefficients, per-type parameters) come from the spec at run time — never hoisted onto the class — so one operator serves every parameterization.

7. Declare the capability contract. An operator lists the spec keys it needs (`REQUIRES_PARAMS`) and the per-type node properties it reads (`REQUIRES_TYPE_PROPS`, resolved along the containment chain). The validator checks these *before* the run, so a missing parameter is a clear message, not a crash deep in a substep.

Construct	Semantics	Example
`general:` — the run and the world (the space + the clock)		
`name`	run identifier	`name: aggregate`
`seed`	RNG seed — fixes determinism	`seed: 0`
`n_frames`, `dt`	ticks to run; the time step	`n_frames: 250`
`boundary`	the space: `periodic` (torus) or `wall` (clamp)	`boundary: periodic`
`world`	domain width W; world is $[0, W]\times[0, 1]$ ($W>1$ longitudinal)	`world: 4.0`
`obstacles`	static walls (rects or `[cx,cy,r]` circles); a BC for the MPM grid + field diffusion	`[[.3,0,.36,.7]]`
Set (S_k — an entity level / ECS entity)		
`n`	size of a top-level set	`n: 100`
`parent`	the set that contains this one — a containment edge $\pi_{A\to B}$ (many sets may share a parent)	`parent: cell`
`per_parent`	children per parent	`per_parent: 100`
`role`	this set's role inside its parent (membrane / cytoplasm / nucleus...)	`role: membrane`
`types`	named sub-populations: fraction + per-type properties	`{soft: {fraction: .5, youngs: 12}}`
Field (a continuum)		
`frame`	discretization (a grid)	`frame: grid`
`res`	grid resolution	`res: 96`
`diffusion`, `decay`	PDE coefficients	`diffusion: 0.1`
`couples_to`	the one set this field exchanges with	`couples_to: cell`
`source`	fixed source region (imposes a gradient)	`source: [.82,.82,1,1]`
`init`	uniform initial fill (self-generated runs)	`init: 1.0`
`navigation`	`geodesic` (precomputed external gradient) or `dynamic` (self-generated)	`navigation: geodesic`
Operator (a system; one per list entry)		
`op`	which registered operator to run	`op: sense`
`at`	selector — the subset it acts on	`at: cell[type=soft]`
`to`/`from`	field an Exchange writes to / reads from	`from: chemical`
⟨params⟩	operator-specific arguments	`gain: 150`
Selector (sub-grammar)		
`set`	every node of the set	`cell`
`set[attr=val]`	nodes matching `attr` (re-checked each tick)	`cell[loaded=1]`
Schedule (sub-grammar; per-tick order)		
operator name	run that operator this tick	`sense`
`[a, b]`	a group run in the same stage	`[motility, sense]`
`aggregate`	builtin: Aggregate↑ (parent = mean of children)	`aggregate`
`name.diffuse`	builtin: advance that field one step	`chemical.diffuse`
`plotting:` (render style only — never affects dynamics)		
`colormap`, `point_size`, `background`	how the sets are drawn	`background: black`

Table 2: The simulation language. Each block is a category with its own home in the spec (Section 8): `general:` is the world and clock, `sets:` are the entities, `operators:` the dynamics, `schedule:` the composition, `plotting:` the style. Integration is implicit (each set is integrated once at the end of a tick), so there is no `integrate` token.

Category	Owns — the question it answers	Home
World	the space: boundary, domain size, obstacles — *"what space do nodes live in?"*	`general:`
Set (entity)	what a node *is*: count, composition (types + per-type properties), initial state — *"what is a node, and how many?"*	`sets:` (+ `@register_entity`)
Operator	the dynamics: an interaction law, its parameters, and what its output *means* (`PREDICTION`, `KIND`) — *"how does state change?"*	`operators:` (+ the registry class)
Schedule	the composition: time step, order, rates — *"when, in what order?"*	`schedule:`, `dt`
Style	rendering only — *"how is it drawn?"*	`plotting:`

Table 3: The categories of the spec. They are orthogonal; a quantity belongs to the one whose question it answers, never to two.

Quantity	Tempting (wrong)	Correct	Why
integration order (1st/2nd derivative)	a field on the particle set	an intrinsic operator property (`PREDICTION`)	it is what the operator's output *means* (velocity vs. acceleration), not what a particle is
friction / damping	an integrator knob or a set field	a `drag` operator ($-\kappa v$)	friction is a force; forces are operators
velocity cap	a particle field	a numerical guard in the integrator	it is about stepping, not the entity
boundary (periodic/wall)	a per-operator flag	the world (`general:`)	the space is shared; all spatial operators must agree
interaction radius, σ, k	the set	the operator that uses them	they parameterize the law, not the node
per-type law params p	the operator's own table	the set's `types:`, *read* by the operator	they are per-*type* (a property of the node's kind) and the inverse-problem target; the type owns, the operator consumes
which column is position / velocity	hard-coded in the operator	the entity state schema (`@register_entity`)	column meaning is a property of the entity kind

Table 4: Category errors and their corrections. The pattern is constant: a process parameter wrongly attached to the thing it acts on.

#	food	speed	gain	drag	rot	youngs	a_max	radius	n
init	120	40	180	2.00	0.30	550	700	0.01	60
1	142	74	379	2.54	0.10	517	454	0.02	46
2	225	106	253	1.25	0.31	36	599	0.01	88
3	298	83	270	0.97	0.48	78	1362	0.01	93
4	423	49	391	1.55	0.18	160	1438	0.01	106
5	525	21	341	2.99	0.40	24	1633	0.01	120
6	553	52	400	0.92	0.56	20	880	0.01	116
7	587	46	342	1.70	0.39	20	988	0.01	120

Table 5: The optimization log (`WINNERS.md`): the initial design and each significantly-better foraging design UCB found, with the eight tuned parameters. Food delivered rises from 120 to 587 ($\sim$5$\times$).